

Unit 1: Designing for Interaction

My Blocks and Natural Language Processing (NLP)

Project: Smart Light Control

Learning Objectives

By the end of this activity, you will be able to:

- Understand what **My Blocks** are and why they are useful.
 - Understand the basic idea of **Natural Language Processing (NLP)**.
 - Create a simple interactive system using **user input and programmed responses**.
 - Use **training data** to teach an NLP text classifier.
 - Use a sprite's **costumes** to show different states of a system.
 - Use **broadcast messages** to communicate between different parts of a program.
 - Test and improve an interactive system.
-

What are My Blocks?

My Blocks allow us to create our own blocks in PictoBlox. Instead of keeping a large program as one long set of instructions, we can group related instructions together and give them a meaningful name.

For example, in our project, we create a My Block named:

Train Data: This My Block contains the instructions needed to prepare and train our NLP text classifier.

Why are My Blocks useful?

My Blocks help us to:

1. Organize our program.
2. Make the code easier to understand.
3. Break a large program into smaller parts.
4. Reuse a group of instructions when needed.
5. Keep related instructions together.

In our project We use My Blocks to organize the **training data** for our NLP model. We have separate My Blocks for:

- **Data for Light ON**
- **Data for Lights OFF**

This makes our training examples easier to organize and manage.

What is NLP?

NLP stands for Natural Language Processing. NLP is a technology that helps computers work with and understand **human language**. Normally, we give instructions to a computer using buttons, menus or commands. With NLP, we can give instructions using **words or sentences**.

Example

The user types: "I want to read."

The NLP system can classify this sentence as: **Light ON**

The user types: "I want to sleep."

The NLP system can classify this sentence as: **Light OFF**

The computer learns how to do this by using **training examples**.

Our Interactive System

Our project is a **Smart Light Control System**.

The robot communicates with the user and asks: "**What do you want me to do?**"

The user types an instruction.

The NLP text classifier checks the instruction and classifies it as: **Light ON** or **Light OFF**

The program then sends a message to the light-bulb sprite.

How does it work?

User gives an instruction

↓

NLP classifies the instruction

↓

Program makes a decision

↓

Broadcast sends a message

↓

Light changes its state

This is an example of **interaction between a user and a digital system**.

Preparing the Sprites

Our project uses two sprites: Sprite 1 – Robot, Sprite 2 – Light Bulb

Sprite 1 – Robot

The robot communicates with the user.

It asks: "**What do you want me to do?**"

Sprite 2 – Light Bulb

The light bulb represents our smart light.

It has **two costumes**:

Costume	Represents
light-bulb	Light ON
light-bulb2	Light OFF

Why do we use two costumes?

A costume allows a sprite to have a different appearance. In our project, the two costumes represent two different **states** of the light:

light-bulb → ON

light-bulb2 → OFF

TRAINING THE NLP MODEL

Why do we need to train the NLP model?

A computer does not automatically know what we mean when we type a sentence. We need to give it examples first. These examples are called **training data**.

Our NLP model learns to classify sentences into two categories:

- **Light ON**
- **Light OFF**

The quality of the training examples affects how well the classifier can classify new sentences.

Creating the Training Data

We will create two groups of examples.

Category 1 – Light ON

Examples:

- It is dark.
- I want to read.
- I want to work.
- I want the light to be on.
- Turn the light on.

These examples teach the classifier: "**These sentences mean Light ON.**"

Category 2 – Light OFF

Examples:

- I am tired.
- I want to sleep.
- I want to watch a movie.
- I have a headache.
- Turn the light off.

These examples teach the classifier: "**These sentences mean Light OFF.**"

Create the Train Data My Block

Create a My Block called: **Train Data**

Inside this My Block, we will perform four important steps.

Step 1 – Reset the Text Classifier

Use:

Reset Text Classifier

This clears any previous training data.

Step 2 – Add Light ON examples

Use the **Data for Light ON** My Block to add sentences that represent the Light ON category.

For example:

"I want to read." → Light ON

"I want to work." → Light ON

Step 3 – Add Light OFF examples

Use the **Data for Lights OFF** My Block to add sentences that represent the Light OFF category.

For example:

"I am tired." → Light OFF

"I want to sleep." → Light OFF

Step 4 – Train the Text Classifier

Finally, use:

Train Text Classifier

This tells PictoBlox to learn from all the examples we have provided.



What is Training Data?

Training data is a collection of examples used to teach a computer how to recognize patterns.

For our project:

Training examples

↓

NLP learns the categories

↓

New sentence

↓

NLP predicts the category

Remember

Good training examples should be:

- Clear
- Relevant
- Different from each other
- Related to the correct category

MAIN PROGRAM

Asking the User

Now that our NLP model is trained, we can create the main program.

The program starts with:

When Green Flag Clicked

This means the program starts when we click the green flag.

Step 1 – Train the Classifier

First, use: **Train Data**

This makes sure that our NLP classifier is trained before the user enters an instruction.

Step 2 – Ask the User

Use the:

Ask and Wait

block.

Ask:

"What do you want me to do?"

The user can type a sentence such as:

"I want to read."

The answer entered by the user is stored by PictoBlox as:

Answer

Step 3 – Find the Class of the Answer

Now use the NLP block:

Get class of answer

This asks the NLP classifier to identify which category the user's sentence belongs to.

The result will be something like:

Light ON

or

Light OFF

Making a Decision with IF–ELSE

Now we need to tell the computer what to do with the result.

We use an **if–else** block.

The logic is:

IF the class of the answer is **Light ON**

→ **Broadcast Lights ON**

ELSE

→ **Broadcast Lights OFF**

In simple words

If the NLP classifier says:

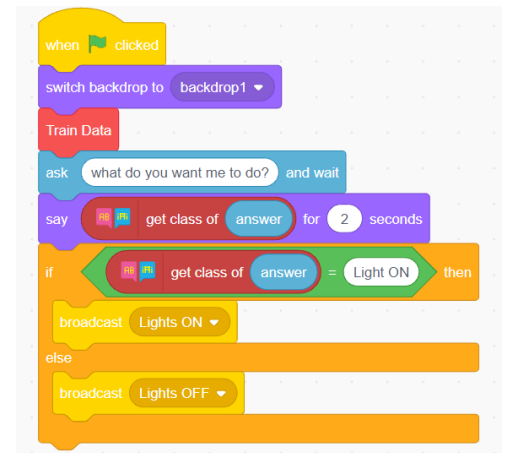
Light ON

the program sends:

Lights ON

If the answer is not classified as Light ON, the program sends:

Lights OFF



What is a Broadcast?

A **broadcast message** allows one part of a program to send a message to another part of the program. In our project, the main program sends one of two messages: **Lights ON** or **Lights OFF**

The light-bulb sprite receives the message and changes its appearance.

Simple idea

Robot / NLP

↓

Broadcast: Lights ON

↓

Light Bulb receives the message

↓

Light Bulb changes to ON costume

Light Bulb Program

Step 1 – Hide the light bulb when the program starts

Use:

When Green Flag Clicked

↓

Hide



Step 2 – Program the Lights OFF message

When the light bulb receives:

Lights OFF

It should:

1. Show itself.
2. Change the backdrop.
3. Change its costume to **light-bulb2**.

Blocks:

When I Receive [Lights OFF]

↓

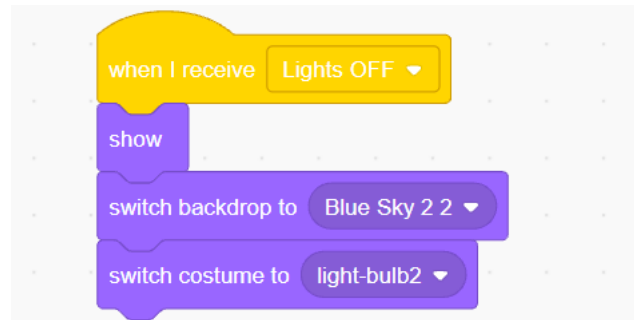
Show

↓

Switch Backdrop to [Blue Sky 2 2]

↓

Switch Costume to [light-bulb2]



Step 3 – Program the Lights ON message

When the light bulb receives:

Lights ON

It should:

1. Show itself.
2. Change the backdrop.
3. Change its costume to **light-bulb**.

Blocks:

When I Receive [Lights ON]

↓

Show

↓

Switch Backdrop to [Blue Sky 2]

↓

Switch Costume to [light-bulb]

